COM3229 Test 1 – Model Answers

PART 1: SQL Queries (EMP / DEPT)

1. Employee number, name, job and salary of all employees.

```
SELECT empno, ename, job, sal
FROM   emp;
```

2. Names and salaries of employees who earn more than 2,000.

```
SELECT ename, sal
FROM   emp
WHERE  sal > 2000;
```

Result: CLARK, KING, JONES, SCOTT, FORD, BLAKE.

3. Names and jobs of employees in department 10 or 30.

```
SELECT ename, job
FROM   emp
WHERE  deptno IN (10, 30);
```

Equivalent: WHERE deptno = 10 OR deptno = 30

4. Names and salaries between 1,500 and 3,000 inclusive.

```
SELECT ename, sal
FROM   emp
WHERE  sal BETWEEN 1500 AND 3000;
```

BETWEEN is inclusive of both bounds, so TURNER (1500), SCOTT and FORD (3000) are included.

5. Name, salary and department number, sorted by department ascending then salary descending.

```
SELECT   ename, sal, deptno
FROM     emp
ORDER BY deptno ASC, sal DESC;
```

6. All details of employees whose names begin with S.

```
SELECT *
FROM   emp
WHERE  ename LIKE 'S%';
```

Result: SMITH, SCOTT. (Oracle string comparison is case-sensitive; data is stored in upper case.)

7. Name, job and department number of managers.

```
SELECT ename, job, deptno
FROM   emp
WHERE  job = 'MANAGER';
```

8. Names of employees who do not work in department 20.

```
SELECT ename
FROM   emp
WHERE  deptno <> 20;
```

!= also works in Oracle.

9. Names, salaries and commissions of employees whose commission is not NULL.

```
SELECT ename, sal, comm
FROM   emp
WHERE  comm IS NOT NULL;
```

Result: MARTIN, WARD, TURNER, ALLEN. TURNER is included because 0 is a value, not NULL. Never write comm != NULL – that comparison is never true.

10. Names and salaries of employees earning more than 1,500 in department 30.

```
SELECT ename, sal
FROM   emp
WHERE  sal > 1500
AND    deptno = 30;
```

Result: ALLEN (1600), BLAKE (2850). TURNER (1500) is excluded because the condition is strictly greater than.

11. Total number of employees.

```
SELECT COUNT(*) AS total_employees
FROM   emp;
```

Result: 14.

12. Lowest, highest and average salary.

```
SELECT MIN(sal) AS lowest_sal,
       MAX(sal) AS highest_sal,
       ROUND(AVG(sal), 2) AS avg_sal
FROM   emp;
```

Result: 800, 5000, 2073.21.

13. Each department number and the number of employees in it.

```
SELECT   deptno, COUNT(*) AS num_employees
FROM     emp
GROUP BY deptno;
```

Result: 10 → 3, 20 → 5, 30 → 6. (Dept 40 has no employees, so it does not appear. To show it with 0, use DEPT LEFT JOIN EMP and COUNT(e.empno).)

14. Each department and its average salary, only where the average exceeds 2,000.

```
SELECT   deptno, ROUND(AVG(sal), 2) AS avg_sal
FROM     emp
GROUP BY deptno
HAVING   AVG(sal) > 2000;
```

Result: 10 → 2916.67, 20 → 2175.00 (dept 30 averages 1566.67, so it is filtered out). HAVING, not WHERE, must be used because the condition is on an aggregate.

PART 2: Patel Car Service & Repair Centres

a) Four key phases of the DBLC applied to Patel Centres [8]

Phase	What it involves	Application to Patel Centres
1. Database initial study	Analyse the company situation, define problems and constraints, set objectives, scope and boundaries.	Study how the three centres currently use manual files for maintenance history, inventory, purchasing, billing, hours and payroll. Problems: duplicated records, lost files, no shared view of a vehicle's history across centres, slow billing and stock control. Objective: one computerised system for all three centres. Scope: vehicles sold by Ngobeni only; boundaries: budget, hardware, staff skills.
2. Database design	Conceptual design (ER model), DBMS selection, logical design (tables, keys, normalisation), physical design (storage, indexes, access paths).	Build an ERD with entities such as CUSTOMER, VEHICLE, SERVICE_RECORD, PART, EMPLOYEE, CENTRE, INVOICE. Normalise to 3NF (e.g. a SERVICE_PART bridge table for the M:N between services and parts). Choose a DBMS (e.g. Oracle/MySQL), define keys and constraints, and index frequently searched columns such as VIN and registration number.
3. Implementation and loading	Install the DBMS, create the database and tables, implement security and integrity rules, and load/convert data.	Create the schema with DDL, set up user accounts and roles (manager, receptionist, mechanic), and convert the existing paper records of all three centres into the database – capturing vehicles, owners, service history and stock levels, cleaning duplicates during loading.

4. Testing and evaluation	Test the database for correctness, integrity, security, performance and recoverability; evaluate it against the objectives; fine-tune.	Test that a service at one centre updates parts stock and generates a correct invoice; that receptionists cannot view payroll; that queries on vehicle history run quickly; and that backup and recovery work. Users at each centre confirm it meets their needs before full rollout.

b) Four major entities with one attribute each [4]

- **CUSTOMER** (vehicle owner) – attribute: *Cust_ID* (also name, phone).
- **VEHICLE** – attribute: *VIN* / registration number.
- **SERVICE_RECORD** (repair/service job) – attribute: *Service_Date* (or Repair_Cost).
- **PART** (spare-parts inventory) – attribute: *Qty_On_Hand* (or Part_No).

Other acceptable entities: EMPLOYEE (Emp_ID, Job_Title), CENTRE (Centre_ID, Location), INVOICE (Invoice_No, Amount), SUPPLIER/PURCHASE_ORDER, TIMESHEET (Hours_Worked).

c) Centralized vs decentralized design; recommendation [4]

Centralized design: one designer/team produces a single conceptual model for the whole organisation in one step. Suited to relatively small, simple problems with few entities and a single data view.

Decentralized design: the problem is split into modules (e.g. per site or per business function), each designed separately by its own team, then the local models are aggregated/integrated into one global model. Suited to large, complex systems with many entities and independent units, but requires resolving conflicts (synonyms, homonyms, differing entity/attribute definitions) during aggregation.

Recommendation: decentralized design. The three centres are independently managed and each has its own procedures, staff and inventory, and the system spans several functional areas (servicing, inventory, purchasing, billing, payroll). Designing each centre's/function's requirements locally captures real practices accurately; the modules are then aggregated into one integrated database owned by Ngobeni so that a vehicle's history is visible at any centre. (Note: this is a *design approach* – the resulting database can still be physically hosted centrally.)

d) Purpose of a data dictionary; two examples [2]

A data dictionary is the “database about the database”: it stores metadata describing every data element, so that designers, programmers and users use consistent names, types and definitions across all three centres, reducing inconsistency and redundancy and supporting design, documentation and maintenance.

Examples of information held:

- Table and attribute names with data types and lengths, e.g. VEHICLE.VIN CHAR(17).
- Constraints and relationships – primary/foreign keys, NOT NULL, permitted value ranges, plus users/access privileges and creation dates.

e) One data-integration problem and solution [2]

Problem – duplicate and inconsistent records: the same customer or vehicle may have been recorded at more than one centre with different spellings, formats or details (e.g. “J. Mokoena” vs “John Mokoena”, different phone numbers or date formats). Merging creates duplicates and conflicting data, so a vehicle's service history is split across records.

Solution: standardise formats and clean the data before loading (data cleansing/de-duplication), match records on a reliable unique identifier such as the vehicle's VIN or customer ID number, assign one surrogate primary key per real-world entity, and enforce uniqueness constraints so duplicates cannot be re-entered.

PART 3: University Database Security

a) Four security threats in the scenario and their impact [4]

- **Excessive privileges (privilege abuse / insider threat):** staff with more access than their job requires can view, alter or leak confidential student, results and financial data – e.g. changing marks or fees, breaching privacy law (POPIA).
- **Unpatched DBMS (known vulnerabilities):** attackers can exploit published flaws to gain unauthorised access, escalate privileges, crash the system or steal data.
- **Inadequate auditing / unreviewed logs:** malicious or accidental activity goes undetected; breaches cannot be investigated or attributed, and there is no accountability or early warning.
- **SQL injection via the web portal:** user input concatenated into SQL lets an attacker alter queries to bypass login, read other students' records, modify results or drop tables – loss of confidentiality and integrity.

b) Countermeasures for three threats [6]

- **SQL injection → parameterised queries / prepared statements (plus input validation):** user input is bound as data, never executed as SQL code, so injected text cannot change the query's structure.
- **Excessive privileges → principle of least privilege with role-based access control:** grant each role only the privileges its job requires and periodically review/revoke access, limiting what any one account (or a compromised one) can see or change.
- **Unpatched DBMS → regular patch management:** apply vendor security patches promptly after testing, closing known vulnerabilities before attackers can exploit them.

(Alternative for auditing: enable database auditing and review logs regularly / use automated alerts, so suspicious activity is detected quickly and users are held accountable.)